

Instruções Gerais de Instalação, Execução e Validação

Este documento apresenta as instruções para instalar as dependências necessárias, configurar o Ollama, executar o **Evidence-First Agent** e, ao final, executar os testes automatizados do projeto.

Importante: siga as etapas na ordem apresentada. O sistema deve ser executado e utilizado **antes** da execução dos testes automatizados.

Observação: Já foram adicionados à pasta data/bronze_raw os dados brutos do dataset “Bird census counts at the Zwin Nature Park” para testar o sistema, baixados do GBIF, portanto você não precisa se preocupar em baixar/descompactar e subir os dados para a pasta, pois já estão lá.

1. Pré-requisito: instalação do Ollama

O sistema utiliza o **Ollama** para executar localmente o modelo de linguagem utilizado no projeto. Antes de executar o sistema, é necessário instalar o Ollama no computador.

Windows

Há duas formas de instalar o Ollama.

Opção 1 - Download

Baixe e instale o Ollama pelo site oficial:

<https://ollama.com/>

Opção 2 - Terminal

Em computadores Windows com **winget** disponível, abra o PowerShell e execute:

```
winget install Ollama.Ollama
```

Após a instalação, recomenda-se reiniciar o terminal.

2. Instalação do Ollama para Python

Com o Ollama instalado no computador, abra um novo terminal e instale o pacote Python utilizado pelo projeto:

```
python -m pip install ollama
```

3. Instalação do modelo utilizado pelo projeto

O modelo de linguagem utilizado pelo sistema é o: llama3.2

Para instalá-lo, execute:

```
ollama pull llama3.2
```

O download pode levar alguns minutos, dependendo da velocidade da conexão e do computador.

4. Reinicie o terminal

Após concluir a instalação do Ollama e do modelo, feche o terminal utilizado anteriormente e abra um **novo PowerShell**.

Isso garante que as configurações e comandos adicionados ao ambiente do sistema sejam reconhecidos corretamente.

5. Verificação da instalação

Antes de executar o projeto, verifique se o Ollama está instalado e funcionando corretamente.

5.1 Verificar a versão do Ollama

Execute:

```
ollama --version
```

O comando deve retornar a versão instalada do Ollama.

5.2 Verificar os modelos instalados

Execute:

```
ollama list
```

O modelo utilizado pelo projeto deve aparecer na lista: llama3.2

5.3 Verificar modelos em execução

Execute:

```
ollama ps
```

Esse comando mostra os modelos atualmente carregados/em execução no Ollama. É normal que a lista esteja vazia antes de o sistema realizar uma solicitação ao modelo.

5.4 Testar a comunicação entre Python e Ollama

Execute o seguinte comando exatamente como apresentado:

```
python -c "import ollama; r=ollama.chat(model='llama3.2',  
messages=[{'role':'user','content':'Responda apenas OLLAMA OK'}]);  
print(r['message']['content'])"
```

O resultado esperado é uma resposta do modelo contendo:

OLLAMA OK

A obtenção dessa resposta confirma que o Python consegue importar o pacote `ollama`, comunicar-se com o Ollama e realizar uma chamada ao modelo `llama3.2`.

6. Executando o Evidence-First Agent

Após concluir e validar a instalação do Ollama, entre na pasta do projeto:

```
cd "caminho\para\evidence-first-agent"
```

Você pode usar um ambiente virtual, caso queira!

Instale as dependências do projeto:

```
python -m pip install -e ".[dev]"
```

Em seguida, execute o projeto utilizando o comando:

```
python -m evidence_agent
```

7. Interação com o LLM

Após iniciar o sistema, faça uma pergunta compatível com as análises suportadas pelo projeto, como por exemplo:

```
"Qual foi o mês com maior riqueza de espécies?"
```

Quando a configuração do Ollama estiver funcionando corretamente, **a resposta pode levar alguns momentos para ser apresentada**. Esse comportamento é esperado. O sistema primeiro realiza as etapas analíticas e de verificação e, posteriormente, utiliza o modelo `llama3.2` para sintetizar a resposta.

Portanto:

Não é esperado que a resposta final apareça instantaneamente.

Uma resposta que apareça **direta e imediatamente**, sem qualquer tempo de processamento, pode indicar que a etapa de geração pelo LLM foi ignorada ou que o sistema utilizou o caminho determinístico/fallback.

para sair da caixa de diálogo é só digitar:

```
exit
```

Depois de rodar todo o sistema pela primeira vez, caso queira apenas abrir a Interface de Linha de Comando para conversar direto com o LLM e fazer mais perguntas, basta rodar apenas:

```
python -m evidence_agent.cli
```

para sair da caixa de diálogo é só digitar:

```
exit
```

8. Relatórios gerados

Os resultados das execuções ficam registrados em arquivos de relatório no formato **Markdown (.md)**.

Após utilizar o sistema, consulte a pasta:

```
reports/
```

Os relatórios gerados ficam armazenados nessa pasta e podem ser abertos com qualquer editor de texto compatível com Markdown. Esses arquivos permitem verificar as respostas e os resultados produzidos pelo sistema após a execução das análises.

9. Execução dos testes automatizados

Importante

Os testes automatizados devem ser executados **somente depois de o sistema ter sido utilizado e a interação com o LLM ter sido verificada**.

Isso permite validar primeiro o funcionamento real da aplicação e, posteriormente, verificar a integridade do código por meio da suíte automatizada.

Com o ambiente virtual ativado, execute:

```
python -m pytest -v
```

O `-v` habilita o modo detalhado do pytest, permitindo visualizar individualmente os testes executados e seus respectivos resultados.

Se tudo correu bem você receberá o seguinte resultados dos testes:

```
tests/adversarial/test_adversarial.py::test_leading_question_does_not_override_observed_result PASSED [ 3%]
tests/adversarial/test_adversarial.py::test_qualitative_presence_is_not_treated_as_quantitative_abundance PASSED [ 6%]
tests/adversarial/test_adversarial.py::test_unsupported_question_is_rejected PASSED [ 9%]
tests/adversarial/test_adversarial.py::test_effort_representations_are_not_equated PASSED [ 12%]
tests/adversarial/test_adversarial.py::test_independent_audit_verifies_engine_claim PASSED [ 15%]
tests/adversarial/test_adversarial.py::test_auditor_rejects_tampered_claim PASSED [ 18%]
tests/adversarial/test_adversarial.py::test_missing_count_is_not_zero PASSED [ 21%]
tests/adversarial/test_adversarial.py::test_effort_claim_explicitly_records_limitation PASSED [ 24%]
tests/adversarial/test_adversarial.py::test_claim_requires_audit_before_being_considered_verified PASSED [ 27%]
tests/adversarial/test_adversarial.py::test_claim_id_is_reproducible PASSED [ 30%]
tests/integration/test_audit.py::test_peak_richness_is_independently_verified PASSED [ 33%]
tests/integration/test_audit.py::test_abundance_is_independently_verified PASSED [ 36%]
tests/integration/test_audit.py::test_effort_claim_is_independently_verified PASSED [ 39%]
tests/integration/test_audit.py::test_auditor_detects_tampered_peak_result PASSED [ 42%]
tests/integration/test_pipeline.py::test_peak_richness_pipeline_completes PASSED [ 45%]
tests/integration/test_pipeline.py::test_abundance_pipeline_completes PASSED [ 48%]
tests/integration/test_pipeline.py::test_effort_pipeline_completes PASSED [ 51%]
tests/integration/test_pipeline.py::test_unsupported_pipeline_is_rejected PASSED [ 54%]
tests/unit/test_evidence_engine.py::test_peak_richness_returns_valid_claim PASSED [ 57%]
tests/unit/test_evidence_engine.py::test_peak_richness_handles_ties PASSED [ 60%]
tests/unit/test_evidence_engine.py::test_abundance_preserves_qualitative_records PASSED [ 63%]
tests/unit/test_evidence_engine.py::test_missing_individual_count_is_not_converted_to_zero PASSED [ 66%]
tests/unit/test_evidence_engine.py::test_effort_claim_preserves_distinct_representations PASSED [ 69%]
tests/unit/test_evidence_engine.py::test_claim_has_reproducibility_metadata PASSED [ 72%]
tests/unit/test_evidence_engine.py::test_claim_id_is_stable_for_same_analysis PASSED [ 75%]
tests/unit/test_quality.py::test_quality_report_exists PASSED [ 78%]
tests/unit/test_quality.py::test_quality_report_has_expected_structure PASSED [ 81%]
tests/unit/test_quality.py::test_quality_report_contains_effort_warning PASSED [ 84%]
tests/unit/test_quality.py::test_quality_report_detects_multiple_protocols PASSED [ 87%]
tests/unit/test_skills.py::test_peak_richness_question_is_routed PASSED [ 90%]
tests/unit/test_skills.py::test_abundance_question_is_routed PASSED [ 93%]
tests/unit/test_skills.py::test_effort_question_is_routed PASSED [ 96%]
tests/unit/test_skills.py::test_unsupported_question_is_not_routed PASSED [100%]

===== 33 passed in 3.47s =====
```